Team 7: Brendan Reed, Elias Miller, Ethan Merritt, Isaiah Wood, Marie McCauley, Zimren Dixon

Vulnerability List & Table of Contents

Google Gruyere Details 1
I. Security Misconfiguration: Debug Information by Elias Miller 2
II. Broken Access Control: Privilege Escalation Through Request Tampering by Elias Miller 3
III. DoS: Overload the Server by Zimren Dixon 4
IV. DoS: Quit the Server by Zimren Dixon 5
V. XSS: Reflected XSS by Brendan Reed 6
VI. File Handling: Directory Traversal by Brendan Reed 7
VII. Cross-Site Scripting: Stored XSS by Isaiah Wood 9
VIII. Cross-Site Request Forgery: Basic by Isaiah Wood 11
IX: Insecure Deserialization: Remote Code Execution via Unsafe Deserialization by Ethan Merritt 12
X. Injection Flaws (Other): Cross-Site Script Inclusion (XSSI) by Marie McCauley 13

Google Gruyere Details

Google Gruyere URL and Instance ID	https://google-gruyere.appspot.com/38511295524546045316659694554591525204 5/

I. Security Misconfiguration: Debug Information by Elias Miller

Part I Description & Impacts

Description: Security misconfigurations can show up in many different ways, but they primarily are focused around accidentally allowing something critical to exist or be accessible with improper controls. For example, this vulnerability displays debug information to any user of the web application.

Root Cause/Problem: The inclusion of the `debug.gtl` file in the public site. No security controls around access to `debug.gtl`. Additionally, all user information (like passwords) is stored in plain text.

Risk Category: High risk.

Technical Impact: The confidentiality of user credentials is compromised.This means that all user accounts are compromised and admin access to the server has also been compromised. This can mean that account details can be changed or availability of the server can be compromised.

Business Impact: With the compromise of all user credentials, the integrity and availability of the application is at risk. Additionally, disclosure of sensitive information may occur, harming the reputation of the business, decreasing userbase, and jeopardizing revenue stream from the application.

STEP 1: Find GTL files
GTL files can be identified in the code base at the following link or by using a web spider within a web proxy application like Burp Suite. `https://google-gruyere.appspot.com/code`
STEP 2: Access file
Enter the following link into the URL. `https://google-gruyere.appspot.com/6161189578936989734579757119275909658 11/debut.gtl`

Video
gruyere_dump.mp4

Part II Mitigation Strategies

Leaving development files and artifacts in production code would be caught through the use of security touchpoints, such as code reviews during development and by risk-based security tests. These tests would check for development files before deployment of the application. This would ensure the removal of the offending file from the code and it would have never reached the public server. Additionally, the code review and risk-based test would have identified that user data is not encrypted on the server. To reduce the impact of disclosure of personal information or the compromise of passwords, all information stored on the server should be encrypted.

II. Broken Access Control: Privilege Escalation Through Request Tampering by Elias Miller

Part I Description & Impacts

Description: Broken access controls are vulnerabilities within the access control mechanism where it does not work as intended or designed. In this vulnerability example, a user can perform a client-side manipulation of get requests and this allows for users to enable admin privileges or create users with admin privileges.

Root Cause/Problem: Lack of server-side validation of authorization at time of request.

Risk Category: Critical

Technical Impact: Account integrity is compromised. Authorization mechanism is bypassed leading to elevated privileges. Any individual can obtain admin privileges. This enables malicious users to perform a denial of service by turning off the server, removing all user accounts, and also data disclosure by using the admin credentials to gain all information hosted on the site.

Business Impact: With the compromise of user authorization integrity, the confidentiality, integrity, and availability of the application and all data it contains is at risk. This creates an extreme financial risk for the organization as the disclosure of sensitive information and disruption of service may occur. This would cause harm to the reputation of the business, potentially decreasing the userbase, and jeopardizing the revenue stream from the application.

STEP 1: Log-In
Log in by providing a username and password. If a username and password has not been created, first Sign-Up, then log-in.
STEP 2: Input Tampered Request
Input the following link into the URL. `https://google-gruyere.appspot.com/6161189578936989734579757119275909658 11/saveprofile?action=update&is_admin=True`
Video

gruyere_priv_esc.mp4

Part II Mitigation Strategies

This could have been mitigated prior to deployment by having a security touchpoint, like a penetration test on the application once it has been developed, or by performing architecture risk analysis and code reviews throughout the development process. This would have ensured that the authorization checks were being performed on the server side at the time of request. This would prevent unauthorized users from having the ability to assign admin privileges.

III.DoS: Overload the Server by Zimren Dixon

Part I Description & Impacts

Description: A denial-of-service (DoS) attack designed to flood a server or network with fake traffic, denying access to applications on the server, making users unable to access the server. This approach to a DoS attack centers on overloading the server with fake traffic, rendering the user unable to access it. The type of known examples of this attack are CVE-2023-44487 and CVE-2020-1104.

Root Cause / Problem: Mitre Attack defines a DoS that overloads the server as a threat, resulting in excessive resource exhaustion or a service crash induced by processes launched by users or scripts that rapidly consume CPU/memory or attempt malformed service interactions. This attack typically occurs at Layers 3, 4, and 7, as these layers are connected to resources to process traffic.

Risk Category: Critical

Technical Impact: Causes the system to be inaccessible, slow performance, or total service downtime. Further, could impact data not properly backup on the database.

Business Impact: The impact of this vulnerability is that it stops the functionality of the server and hinders the connection of the new users, and stops current users from maintaining their connection.

Video of Overload
Dos_Overload.mp4
Walkthrough
Step 1: Create a new account on the website, setting the username as resources and the password as root. Due to the nature of this website, it lacks proper system-enforced access controls and connection to administrative functions, such as resources. This allows me to create a username, resources, and a password root, and access privileges without me, the user, having the clearance. **Step 2: Create a file titled manage.gtl and put in it: [|include:menubar.gtl|]DoS[|/include:menubar.gtul]|** In a file I created called manage.gtl, I will incorrectly add a menubar with the text DoS; however, due to the syntax errors, it will not work. **Step 3: Then, under upload in the account I created called resources** Due to the lack of system-enforced access controls and confusion between user and administrative functions, I can upload my own version of the manage.gtl file. This allows me to override the existing file, causing a crash and shutting down of the server. **Step 4: Once that is done, the server will overload and crash.**

Thus, being overloaded, it will have to be reset to correct the issue.

Part II Mitigation Strategies

Mitre Attack states for overload security touchpoint, under AN1165, that a mitigation strategy for repeated monitor resource usage issues is to address the causes of CPU and memory spikes. These spikes can show up as high request volumes, long handle locks, or frequent crash recoveries. The mitigation process includes both detecting these problems and taking corrective action. Here are the tools and approaches for mitigation.

Security touchpoint: To protect against a DoS Overload the Server attack, teams should approach the matter from multiple angles, including Network-Layer Defenses and Application-Layer Protections. Network-layer defenses against DoS that would have prevented and stopped this attack include firewalls, which block access from suspicious IPs that do not meet firewall requirements, making them blocked by default. Next is an Access Control List, which would monitor traffic flow and restrict access for users based on their IPs and network activity, but cannot block suspicious traffic patterns for users or admins.

On the Application layer, for protection against DoS via input validation, which enabled proper checks of incorrect or malicious input of the service. Done by defining the input string via the length limits or format type, and sanitizing user inputs via removal of inputs such as Resources, and of course, insertion of codes to replace functional code. Lastly, if Zero-Trust were applied to the website, the exposure and attack surface would have been greatly reduced, as attackers would not be able to access components with admin privileges. The application of peer review of the webserver code would have removed key components of this attack.

IV. DoS: Quit the Server by Zimren Dixon

Part I Description & Impacts

Description: This vulnerability is a type of DoS that targets the application via software, exhausting the system resources on those services that are hosted, or exploiting the system to cause them to crash constantly. The type of known examples of this attack are CVE-2019-19781 and CVE-2022-22965.
Root Cause / Problem: This vulnerability is caused by a software weakness that a threat actor exploited to shut down the system and render it inaccessible to users and admins.
Risk Category: Critical
Technical Impact: Techniques adversaries use to disrupt availability or compromise integrity by manipulating business and operational processes. Further, it could impact data that is not properly backed up in the database.
Business Impact: This vulnerability disrupts server functionality, prevents new users from connecting, and disrupts current users' connections. Further, because this attack is directly caused by a threat actor targeting the software of a server, it is often associated with distraction, hacktivism, and extortion.

Video of Server Quit
Dos_quit.mp4
Walkthrough
Step 1: Review code within the website, determine where the gaps in the code and if protection were not taken.

For this first step, I reviewed all of Google Gruyere, which has all the functional files needed to operate. After some review, I noticed that manage.gtl has some key issues in the server operations.
Step 2: In manage.gtl, there are no safeguards on quit the server:
href='/{{_unique_id}}/quitserver'>Quit the server</a></p>
This part of the code has no safeguards against non-admin access when using this command.
Step 3: Put the Google Gruyere address in and add /quitserver for our instance
The server has been quit and must be reloaded without the /quitserver to be active again

Part II Mitigation Strategies

One of the main mitigation strategies for this vulnerability is to Filter Network Traffic. This approach allows CISOs to control network boundaries by blocking source addresses that initiate attacks, targeted ports, and specific transport protocols. Filtering inbound traffic allows organizations to block malicious source IP addresses responsible for abnormal request volumes, restrict access to targeted ports exploited during attacks, and limit or disable vulnerable transport protocols that contribute to resource exhaustion.

Security Touchpoint: To protect against the Quit the Server DoS attack, teams should focus on the web interface and identify poor user access handling. The focus is on best practices to ensure server availability. One method is using ACLs to monitor traffic and restrict access to sensitive endpoints, preventing unauthorized users from reaching the server and shutting it down. Further, Exhaustion Flood Detection Strategy to check for burst of incomplete TCP handshakes or spoof of SYN or ACK packets causing an overload of network.

Following, verify that only admins can access functions like server quit via the URL or other methods, and apply a Zero-Trust framework to all aspects of the code, ensure that administrative functions and inputs are accessible only to administrators. To reduce the attack surface, ensure that users cannot access administrative components or perform actions such as server shutdown without proper authentication and authorization. Lastly, peer review of the webserver code would have removed aspects of this attack from occurring.

V. XSS: Reflected XSS by Brendan Reed

Part I Description & Impacts

Description: A Reflected Cross-Site Scripting (XSS) attack occurs when malicious client-supplied input is immediately returned by a web application in an HTTP response without proper validation. The script executes in the victim's browser within the trusted site context, typically delivered via crafted URLs or web server actions on form submissions. This might appear similar to a Stored XSS attack, however it is unique because the commands are reflected back onto the webserver in the URL it calls, not executing a command already present.

Root Cause/Problem: The vulnerability stems from improper input validation and trusted output. Applications trust user-supplied data and dynamically embed it into HTML, JavaScript, or attributes without sanitization. Lack of secure coding practices, insufficient filtering, and failure to implement contextual escaping enable attackers to inject executable scripts.

Attacks That Target This Vulnerability: Common attacks include session hijacking, credential theft, phishing overlays, malicious redirects, and drive-by malware delivery.

Risk Category: Reflected XSS falls under injection vulnerabilities and is categorized within web application security risks (e.g., OWASP Top 10). It is typically considered high risk when sensitive

data or authenticated sessions are exposed. Exploitation usually requires user interaction.

Technical Impact: Consequences include session token theft, unauthorized actions performed as the victim, browser exploitation, and data exfiltration. Attackers may also bypass protections by executing scripts within the trusted domain, undermining client-side security controls.

Business Impact: Business risks involve reputational damage, loss of customer trust, regulatory exposure, incident response costs, and potential legal liability. Successful exploitation can lead to account compromise at scale, fraud, and service disruption, especially for customer-facing portals.

SCREEN RECORDED EXPLOIT IN ACTION
BRREED_GRUYERE_REFLECTED_XSS.mp4
Step-by-Step Process
Crafting a reflected XSS attack involves creating a link that will be fed back into the server by the web browser. For that reason we identify an existing platform link that will execute our crafted URL. Step 1: Create user account & signin. Step 2: In the user profile area, set a value in the unsanitized “Homepage” field to contain web server instructions executed by the host and sent to the attacker or victims browser when executed. Step 3: Access these instructions via ‘My Snippits’ → ‘My Site’

Part II Mitigation Strategies

Mitigation strategies against reflected XSS include strict input validation, contextual output encoding, and implementing Content Security Policy (CSP) headers to restrict script execution. Web application firewalls help detect and block malicious payloads, while secure development practices prevent unsafe data reflection. Using HTTPOnly and Secure cookie flags protects session tokens from theft. Regular code reviews and penetration testing further reduce exposure. These measures safeguard web server integrity and protect sensitive customer and organizational data from unauthorized access, manipulation, and exfiltration.

Security Touchpoints: Code Review, Architectural Risk Analysis, and penetration testing would have prevented reflected XSS vulnerabilities before deployment. Architectural Risk Analysis would identify insecure data flows where request parameters are directly embedded into dynamic responses without proper encoding

VI. File Handling: Directory Traversal by Brendan Reed

Part I Description & Impacts

Description: Directory Traversal (Path Traversal) is a file handling vulnerability where an attacker manipulates file path input to access files outside the intended directory. By injecting sequences like `../`, the attacker can traverse the server’s filesystem and retrieve sensitive files not meant to be publicly accessible. Additionally, if an attacker is aware of folders and files exposed on the webserver either through reconnaissance or CVEs, they may access folders that are meant to be hidden from view.
Root Cause/Problem: The root cause is improper validation or sanitization of user-supplied file

path input. Applications dynamically construct file paths without restricting traversal sequences or enforcing strict directory constraints. Failure to implement canonicalization checks and secure file access controls allows attackers to escape the intended directory structure.

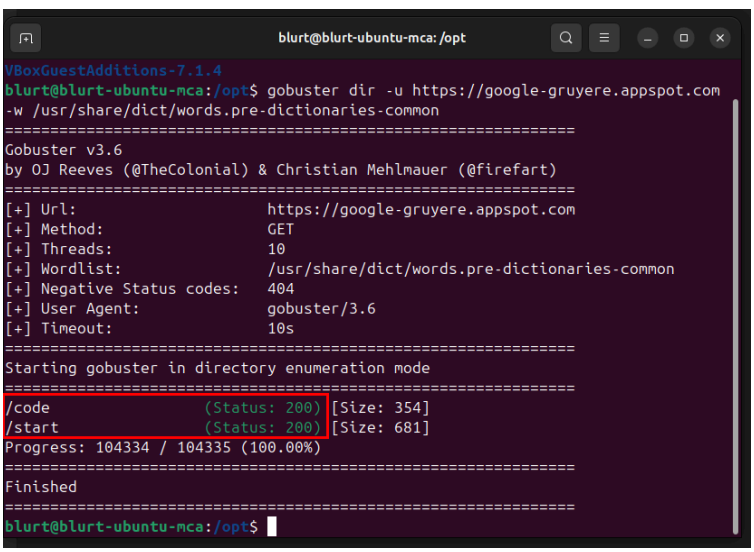
Attacks That Target This Vulnerability: Attackers exploit directory traversal to access sensitive system files (e.g., configuration files, password hashes, API keys), application source code, or user data. In severe cases, traversal can enable remote code execution if attackers access executable scripts or upload directories with improper permissions.

Risk Category: Directory Traversal is categorized as an injection and access control vulnerability under web application security risks (e.g., OWASP Top 10). It is typically considered high severity because it may expose confidential system-level files and compromise the integrity of the host environment.

Technical Impact: Technical impacts include unauthorized file disclosure, exposure of credentials, system configuration leakage, and potential privilege escalation. Attackers may gain insight into server architecture, database connections, or internal APIs, enabling follow-on attacks such as lateral movement or remote code execution.

Business Impact: Business consequences include data breaches, compliance violations, regulatory penalties, and reputational harm. Exposure of sensitive customer or operational data can result in financial loss, incident response costs, and erosion of stakeholder trust, particularly for organizations handling regulated or proprietary information.

Step-by-Step Process



STEP 2: WRITE OUT YOUR STEP HERE
To perform directory traversal, we first need to perform reconnaissance to build intelligence of what directories exist on the web server and are accessible. There are numerous ways to accomplish this. I chose to use *dirbuster* and a default Ubuntu wordlist to search for available directories. This result found /code, /start, and /robots.txt. Directing our browser to the /code subdirectory brings up areas of the webserver which should not typically be visible. This is where we find **secret.txt** and the challenge flag behind Gruyere.

Part II Mitigation Strategies

Mitigation strategies include strict server-side input validation, path canonicalization, and rejecting traversal patterns such as "../" before file processing occurs. Applications should enforce allowlists of approved directories and use secure file-handling APIs that prevent direct user control over filesystem paths. Implementing least-privilege permissions limits access even if exploitation is attempted. Web application firewalls, logging, and continuous security testing further reduce exposure. These controls protect organizational (ROG) systems and customer data from unauthorized disclosure and compromise.

Security Touchpoints: Code Review, Architectural Risk Analysis, and penetration testing would prevent Directory Traversal vulnerabilities before deployment. These points would aim to identify design flaws where user-controlled input directly interacts with file system operations

VII. Cross-Site Scripting: Stored XSS by Isaiah Wood

Part I Description & Impacts

Description: NIST defines cross-site scripting (XSS) as a potential weakness in web applications that enables attackers to inject client-side scripts into a web application. When other users visit the affected page, their browsers execute the script with the same permissions as the website itself, compromising security. In addition, MITRE CWE-79 says that stored XSS involves malicious data being saved in databases, message forums, or other data storage systems. The scripts persist and affect other users.

Root Cause/Problem: NIST says this is a vulnerability in web applications that display user-supplied data from requests or forms without escaping or sanitizing it to prevent execution.

Risk Category: High. MITRE ranks XSS #1 as the most dangerous software weakness in 2025 CWE.

Technical Impact: Arbitrary script execution that may compromise confidentiality, integrity, and availability.

Attacks commonly steal confidential information, such as session cookies. Attacks can compromise the integrity of the user's machine if they use XSS to install Trojan horse programs. Malicious Scripts may disable users' machines, impacting availability. MITRE

Business Impact: Incident recovery costs, reputation damage, and operational disruption.

Incident recovery costs include restoring to trusted baselines, patching vulnerabilities, and legal expenses. Incidents tarnish a company's reputation and deter future business. Lastly, operational disruption results in lost revenue and productivity. Baldwin

STEP 1: Weaponize and Deliver Snippet

Attacker submits malicious code into the new snippet page.

```
<a ONMOUSEOVER="alert('proof of XSS exploit')" href="#">read this!</a>
```

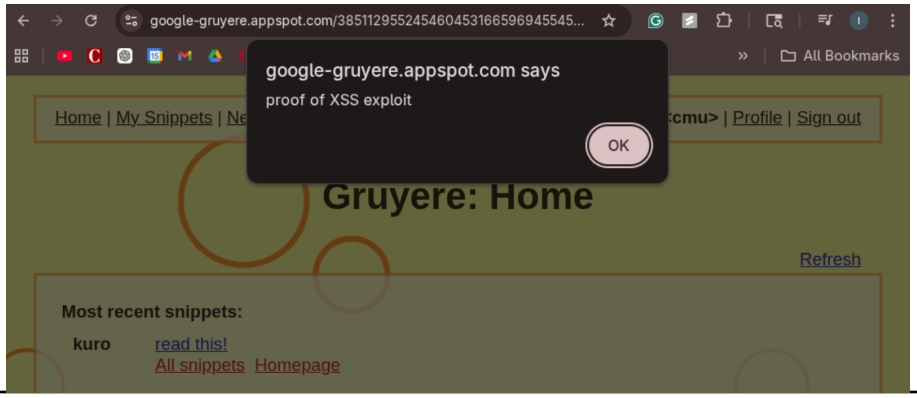
The ONMOUSEOVER command triggers the payload, in this case its “alert(‘proof of XSS exploit’)”.
The malicious code is stored in the Gruyer webpage, and other users will see it.

STEP 2: Victim Views Compromised Webpage

An unsuspecting victim visits the affected page. They see the malicious snippet, “read this!”.

STEP 3: Browser Executes XSS Payload

When the victim hovers over “read this”, the payload is executed. For this demonstration, the payload is an alert “proof of XSS exploit”. The payload could also be a redirection to a phishing page.



Part II Mitigation Strategies

MITRE recommends mitigating stored XSS by using output encoding and input validation. Output encoding converts data to a safe format before displaying, preventing malicious code from executing. Proper output encoding depends on the context language (e.g. HTML, JavaScript, CSS, or URLs). Input validation defines rules for user input (e.g., length, strings, acceptable value ranges). If the user input doesn’t pass the rules check, it's rejected or transformed into an input that does.
Security Touchpoints: Code Review, Architectural Risk Analysis, and penetration testing would have prevented stored XSS vulnerabilities before the code was implemented.

Architectural Risk Analysis would have identified design flaws in storing user input and mapped common vulnerabilities, such as XSS, to them. And, it would have identified the assumptions underlying trust in user input. Next, the code review would have identified overlooked flaws. The peer review should spot unsafe coding practices that trust user input and identify the lack of output encoding. Lastly, the web app penetration testing would have shown poor data handling.

VIII. Cross-Site Request Forgery: Basic by Isaiah Wood

Part I Description & Impacts

Description: OWASP says Cross-Site Request Forgery (CSRF) is a web application vulnerability that tricks a victim's browser into performing unintended actions (e.g. transferring money, changing email addresses, or deleting snippets). The attack requires a victim to authenticate on a site vulnerable to CSRF. At the same time, the victim's machine executes a script (a forged request) provided by the attacker (usually via phishing, malicious links, or compromised websites). The script performs the action using the victim's authenticated session. The server processes the request as if it were from the user because browsers automatically include session cookies with requests.

Root Cause/Problem: A vulnerable web app's architectural design assumes that if a request has a valid session cookie, it is real traffic and is trusted. Also, it does not validate the request origin. This allows attackers to exploit the browser's automatic cookie handling. OWASP

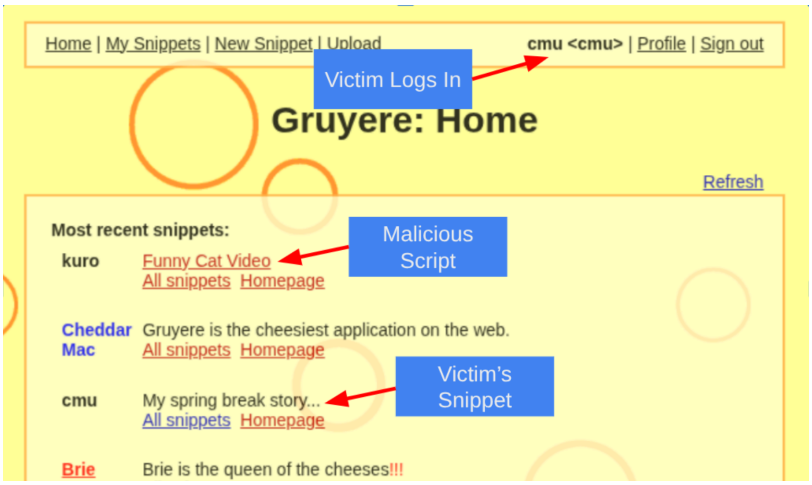
Risk Category: High. MITRE ranks CSRF as the 3rd-most dangerous software weakness in 2025.

Technical Impact: Bypass authentication and execute commands with the victim's privileges. With user privileges, they can modify data or delete files. With Admin privileges, they can take full control of the application, uninstall it, or launch attacks against the app's users. MITRE

Business Impact: Incident recovery costs, reputation damage, and operational disruption.

Incident recovery costs include restoring to trusted baselines, patching vulnerabilities, and legal expenses. Incidents tarnish a company's reputation and deter future business. Lastly, operational disruption results in lost revenue and productivity. Baldwin

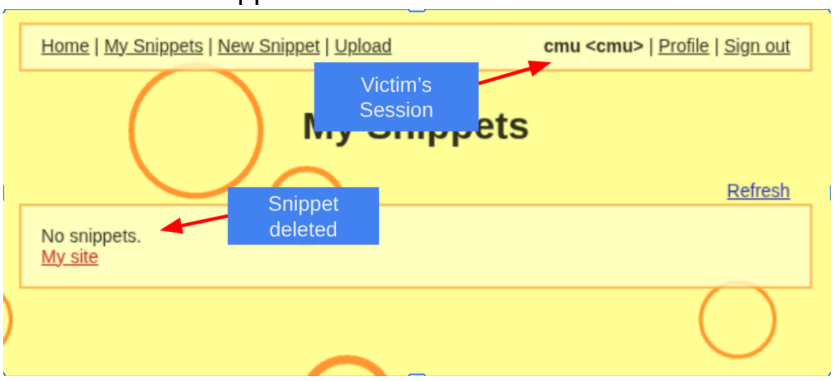
STEP 1: Weaponize and deliver snippet
The attacker submits malicious code into the new snippet page. `<a href="https://google-gruyere.appspot.com/385112955245460453166596945545915252045/deletesnippet?index=0">Funny Cat Video</a>` When users click on the link, their web browser uses their authenticated session to execute the payload. In this case, it deletes their first snippet
STEP 2: User execution and action on objective



The user authenticates on Gruyer, which is vulnerable to CSRF. When they click on the “funny cat video”, their browser executes the forged request. The server processes it because the browser automatically includes session cookies.

STEP 3: Action on objective

The user’s first snippet is deleted



Part II Mitigation Strategies

Description

MITRE mitigates XSRF with the unique nonce for each form and the double-submitted cookie method to verify request intent. The nonce method creates an unpredictable value that put in each form and the server checks for a match at submission. The double submitted cookie method compares a random value stored in a cookie with the corresponding value submitted as a form parameter. Same origin policy prevents attackers from reading these values. Also, MITRE says XSS must be prevented before these methods because XSS can bypass them.

Security Touchpoints: Code Review, Architectural Risk Analysis, and penetration testing would have prevented CSRF vulnerabilities before the code was implemented. Architectural Risk Analysis would have identified design flaws in browsers that automatically send cookies and mapped common vulnerabilities, such as CSRF, to them. And, it would have identified the assumptions underlying trust in session cookies. Next, the code review would have identified

overlooked flaws. The peer review should spot the unsafe coding practices in trusting session cookies, identifying the lack of request origin validation. Lastly, the web app penetration testing would have revealed weaknesses in the high trust placed in session cookies.

IX. Remote Code Execution by Ethan Merritt

Part I Description & Impacts

Description: Remote Code Execution (RCE) occurs when an attacker can cause the server to run attacker-controlled code. In Gruyere, RCE is possible not because of a single bug, but because of a chain of weaknesses. First, an unsafe file/path handling bug allows an attacker to write or replace server-side files outside the intended upload/resource directory. Second, an exposed restart mechanism lets anyone (or at least a low-privileged user) force the application to quit and reload its code. When combined, these flaws allow an attacker to overwrite a server-loaded Python module and then restart the application so that the modified code is executed under the server's identity. This effectively gives the attacker the ability to change how the application behaves at run time and to execute arbitrary logic on the server.

Root Cause/Problem: A chain of weaknesses: (1) unsafe file/path handling that allows writing or replacing server-loaded files and (2) an exposed restart mechanism that reloads modified code.

Attacks That Target This Vulnerability: Once an attacker can reliably overwrite code and trigger a restart, several classes of attacks become possible.

- Web shells / backdoors – Injecting logic that gives the attacker persistent remote access (e.g., hidden admin endpoints, command execution hooks).
- Credential theft and lateral movement – Modifying code to log credentials, session cookies, or database connection strings, then exfiltrating them to pivot deeper into the environment.
- Data exfiltration and service sabotage – Altering business logic to leak sensitive data (profiles, messages, secrets) or to corrupt/erase application data and disrupt service.

Risk Category: Critical

Technical Impact: From a technical perspective, this vulnerability enables arbitrary code execution under the same privileges as the Gruyere web application process. Because the attacker can overwrite server-loaded files and force the application to restart, any injected logic would execute as part of the normal application lifecycle. This means the attacker could potentially access sensitive configuration files, database connections, user data, and internal application logic. They could modify stored data, introduce persistent backdoors, capture credentials or session cookies, or alter how the application processes requests. Since the web server typically operates with access to critical application resources, successful exploitation effectively compromises the integrity and confidentiality of the entire application environment.

Business Impact: From a business perspective, a successful remote code execution vulnerability represents a critical security failure with significant operational and reputational consequences. An attacker gaining control over the application could expose sensitive user information, disrupt service availability, or tamper with stored data, all of which could erode user trust and damage the organization's reputation. Depending on the nature of the data involved, such a breach could also trigger regulatory reporting requirements, legal liability, and financial penalties. Additionally, responding to an RCE incident would require substantial incident response efforts, forensic analysis, potential system rebuilds, and ongoing monitoring, resulting in increased costs and possible service downtime. In a real-world production environment, this

type of vulnerability would likely be classified as a high-severity security incident.

Step-By-Step Process
Step 1: I reproduced the path traversal vulnerability to demonstrate that the application does not properly validate file paths, allowing access to files outside the intended directory structure. This confirmed arbitrary file read/write capability. **Step 2:** I verified that existing server-side files could be overwritten using the same file handling weakness, proving that user-controlled input can modify files within the application's directory tree. **Step 3:** I confirmed that Gruyere loads its Python modules and templates from disk at startup, meaning any modified server-loaded file could influence application behavior. **Step 4:** I triggered the exposed restart ("quit server") endpoint, forcing the application to terminate and reload its code from disk. **Step 5:** By combining arbitrary file overwrite with the restart mechanism, I demonstrated a Remote Code Execution chain in which modified server files are reloaded and executed under the application's process context.
STEP 2: WRITE OUT YOUR STEP HERE
https://drive.google.com/file/d/1CeHUHvIgqiuhcYszzT8T1rRFP1NEVt5S/view?usp=drive_link

Part II Mitigation Strategies

To mitigate this vulnerability, the application must enforce strict separation between user-controlled data and executable server code while implementing robust input validation and authorization controls. All file paths derived from user input should be canonicalized and validated against a strict allow-list to ensure they remain within approved directories after normalization. User uploads should be stored in a dedicated, non-executable directory that is completely separate from application code and resources, and the web application process should follow the principle of least privilege so it does not have permission to modify its own source files. Administrative or restart endpoints must require strong authentication and proper role-based authorization, and security checks should be implemented in a way that cannot be bypassed through case manipulation or similar logic flaws. Together, these measures reduce the likelihood that a file-handling bug can escalate into full remote code execution and significantly strengthen the overall security posture of the application.

X. Injection Flaws (Other): Cross-Site Script Inclusion (XSSI) by Marie McCauley

Part I Description & Impacts

Description: The XSSI vulnerability allows sensitive data leakage across-origin or cross-domain boundaries in a victim's web browser. Like CSRF, XSSI relies on the browser automatically attaching the victim's session cookies to cross-origin requests. However, unlike CSRF, which tricks the victim's browser into executing a malicious action without the attacker seeing the user's response, XSSI uses JavaScript to trick the browser into loading and revealing sensitive information (OWASP, n.d.), such as a user's API key.

Root cause: Resources included within the HTML <script> tag are exempt from the same-origin policy (SOP), which allows a browser to fetch and execute cross-origin requests automatically. SOP is a security rule that prevents scripts on one origin from interacting with another origin.

SOP protects users from malicious sites that may use Javascript to read data from third-party websites and relay information back to the attacker (Mozilla, 2025). However, cross-origin requests are sometimes necessary, such as when a web app in one domain needs to interact with a third-party API. So, mechanisms like cross-origin resource sharing (Portswigger, n.d.) and JSONP (Hyland, 2025) provide SOP bypassing capabilities. But if an attacker creates their own malicious webpage in HTML, then they can include a <script> tag that points to a sensitive endpoint to the victim's vulnerable site. See the example code screenshot in the Step-by-Step Process Demo for an example.

Technical Impact(s): Session hijacking, credential theft, further reconnaissance which can be used to facilitate other cyberattacks and vulnerability exploits

Business Impact(s): Regulatory fines and reputational damage because of data breach

Risk Category: Medium-High (based on Tenable, Cobalt.io, MITRE listing as top CWE 2025)

Step-by-Step Process Demo Video
Marie_McCauley_Google_Gruyere_XSSI_Demo.mp4 **Intro**: 0:00 - 04:46: Vulnerability Explanation & Overview 04:47 - 08:17: Setup requirements: Web browser (minimum 2 tabs), Text Editor, Command Prompt - Optional: disable browser security configurations (to allow exploit to run) **Step 1 -** 08:18 - 09:07: Create an account on Google Gruyere page with a username and password **Step 2 -** 09:08 - 09:32: While logged in on your account, click Profile, then type in a private snippet **Step 3 -** 09:33 - 10:31: - Add feed.gtl to URL **Step 4 -** 10:32 - 11:42: Take exploit script from Google Gruyere's website and put it into a Text Editor Replace the script with your own instance ID. Then, save the script as a .html file.

gruyere_xssi_script.html

```
<script>
function _feed(s) {
  alert("Your private snippet is: " + s['private_snippet']);
}
</script>
<script src="https://google-gruyere.appspot.com/385112955245460453166596945545915252045/feed.gtl"></script>
```

Step 5 - 11:43 - 12:14: Make sure your .html file is saved in your C Drive. Go to Command Prompt. Ensure you're in the folder where your .html file is saved. C:\Users\your_name> **cd Documents** (or wherever your folder is)
Step 6 - 12:15 - 13:00: Start an HTTP server on localhost with command **python3 -m http.server**
Step 7 - 13:01 - 13:46: Run the .html file. You can either drag/drop the file to the browser or use the url http://localhost:8000/your_html_filename.html
Final - 13:47 - 14:15 Verify your exploit ran correctly by going to your browser's Developer Tools -> Response Tab to see the sensitive information
Disclaimer - 14:16 - 17:01 Why isn't this particular exploit fully working on current-day browsers

Part II Mitigation Strategies

Mitigating XSSI requires a two-prong approach at the server and client levels. On the server side, developers should avoid using JSONP to prevent same-origin policy bypasses, exclude sensitive information in Javascript files, prepend responses with)]}'n to break direct script execution, and include anti-CSRF tokens to block unauthorized requests (Pentester Lab, n.d.). These vulnerabilities can be identified and accounted for in the code review and penetration testing touchpoints. Developers should instead use JSON or HTML content types so responses are subject to a browser's same-origin policy. They can also use JSON URLs instead of embedding sensitive data directly into Javascript files (Sallam, 2025). These can be set at the security requirements touchpoint.

At the client level, browsers should be set to authentication cookies with SameSite=Strict to prevent cookies from being sent in any cross-site requests, and enabled with X-Content-Type-Options: nosniff header to prevent browsers from guessing a content type to and executing unintended scripts (Hailperin, 2016). These requirements can be set at the security requirements touchpoints and assessed via risk-based security testing.

Google Gruyere Exploits Videos

Videos